

Лекция 5. Работа с датчиками.

- Использование акселерометра, датчиков магнитного поля, освещенности.
- Жизненный цикл процессов, сохранение состояния приложения.
- Корутины kotlin
- Многопоточное приложение, синхронизация

Использование акселерометра, датчиков магнитного поля и освещённости

Датчики в Android предоставляют возможность взаимодействовать с физическим миром, считывая данные о движении, ориентации, окружающей среде и других параметрах. Основные датчики, используемые в приложениях:

Motion Sensors (Датчики движения)

- **Акселерометр** — измеряет ускорение устройства, включая гравитацию.
- **Гироскоп** — измеряет угловую скорость вращения устройства вокруг своих осей.
- **Датчик магнитного поля (Магнитометр)** — определяет напряжённость магнитного поля, часто используется для навигации (электронный компас).

Environmental Sensors (Датчики окружающей среды)

- **Датчик освещённости** — считывает уровень освещения окружающей среды.
- **Датчик температуры** — измеряет температуру окружающего воздуха (обычно внутри корпуса устройства).
- **Датчик влажности** — измеряет относительную влажность окружающего воздуха.

- **Датчик атмосферного давления (Барометр)** — измеряет атмосферное давление, может использоваться для определения погоды или высоты над уровнем моря (альтиметр).

Position Sensors (Датчики положения)

- **GPS (Глобальная система позиционирования)** — определяет географические координаты (широту, долготу) и высоту устройства с помощью спутниковых сигналов.
- **NFC (Near Field Communication)** — обеспечивает беспроводную связь на очень коротком расстоянии, используется для идентификации, платежей и обмена данными (хотя технически это скорее технология связи, часто относится к датчикам положения/контекста).

Specialized Sensors (Специализированные датчики)

- **Микрофон** — преобразует звуковые волны в электрические сигналы (аудиовход).
- **Камера** — захватывает изображения и видео (оптический сенсор).
- **Датчик приближения** — определяет наличие физического объекта в непосредственной близости от устройства (обычно используется для отключения экрана во время разговора).
- **Барометр** (часто дублируется в Environmental, но также важен для определения высоты - альтиметр).

Общий алгоритм работы с датчиком начинается с **физического взаимодействия**: датчик постоянно подвергается воздействию определенного физического явления из окружающей среды (такого как ускорение, свет, температура, магнитное поле, звук, давление или близость объекта). Это явление **воздействует на чувствительный элемент** (сенсор) внутри датчика, вызывая в нем измеримые изменения физических свойств (например, изменение сопротивления, емкости, напряжения, частоты колебаний или тока). Далее происходит **первичное преобразование**: эти внутренние физические изменения преобразуются в слабый электрический сигнал (часто аналоговый), который напрямую отражает интенсивность

воздействующего явления. Если сигнал аналоговый, а система цифровая, наступает этап **аналого-цифрового преобразования (АЦП)**: аналоговый сигнал поступает на АЦП, который дискретизирует его во времени и квантует по уровню, превращая в последовательность цифровых значений (сырых данных, raw data), понятных микропроцессору. Эти сырые данные обычно требуют **предварительной обработки и калибровки**: микроконтроллер или специализированная схема (часто внутри самого датчика) применяет к цифровым значениям калибровочные коэффициенты (для компенсации погрешностей и смещений), выполняет фильтрацию (например, для удаления высокочастотного шума) и может проводить базовые вычисления (например, усреднение или компенсацию температурного дрейфа). Полученные **обработанные данные** теперь являются более точным цифровым представлением измеряемой физической величины (например, значение ускорения в g, температуры в °C, уровня освещенности в люксах). Эти данные **передаются** по интерфейсу связи (такому как I²C, SPI, UART) в основную вычислительную систему (процессор устройства) или сохраняются в буфере. На уровне приложения или операционной системы происходит **финальная интерпретация и использование**: полученные значения могут подвергаться дальнейшей, более сложной обработке (например, интегрирование ускорения для получения скорости, вычисление ориентации по данным акселерометра и гироскопа, определение координат по GPS), визуализироваться (вывод на экран), использоваться для принятия решений (например, включение подсветки при низкой освещенности, блокировка экрана при поднесении к уху) или передаваться в другие системы. Этот цикл измерения, преобразования, обработки и передачи данных повторяется с заданной датчиком или системой частотой дискретизации, обеспечивая непрерывный или периодический мониторинг окружающей среды или состояния устройства.

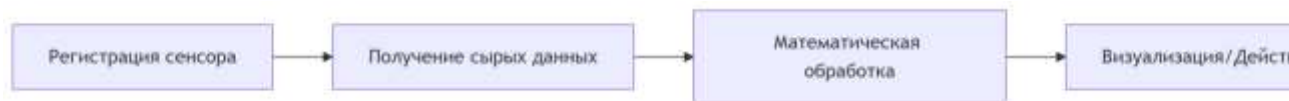


Рисунок 1. Общий алгоритм работы с датчиками

Пример 1: Использование акселерометра для определения движения

// Объявление активности, которая реализует интерфейс `EventListener` для получения данных датчика

```
class SensorActivity : AppCompatActivity(), SensorEventListener {
```

// Менеджер датчиков, который предоставляет доступ к системным датчикам

```
private lateinit var sensorManager: SensorManager
```

// Переменная для хранения акселерометра. Может быть null, поэтому используется тип `Sensor?`

```
private var accelerometer: Sensor? = null
```

// Метод, вызываемый при создании активности

```
override fun onCreate(savedInstanceState: Bundle?) {
```

```
    super.onCreate(savedInstanceState)
```

```
    setContentView(R.layout.activity_sensor) // Установка макета
```

// Получение системного сервиса датчиков

```
sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
```

// Получение акселерометра. Если устройство не поддерживает этот датчик, `accelerometer` останется null

```
    accelerometer =
```

```
    sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
```

```
}
```

// Метод, вызываемый при возобновлении активности (например, после возвращения на экран)

```
override fun onResume() {
```

```
    super.onResume()
```

// Проверка наличия акселерометра перед регистрацией слушателя

```
    accelerometer?.also { sensor ->
```

// Регистрация слушателя с заданной частотой обновления

```
    (SENSOR_DELAY_NORMAL)
```

```
    sensorManager.registerListener(this, sensor,
```

```
    SensorManager.SENSOR_DELAY_NORMAL)
```

```
}
```

```
}
```

// Метод, вызываемый при паузе активности (например, при переходе к другому экрану)

```

override fun onPause() {
    super.onPause()
    // Отмена регистрации слушателя для экономии энергии и предотвращения
утечек памяти
    sensorManager.unregisterListener(this)
}

// Метод, вызываемый при изменении данных датчика. Срабатывает многократно
при движении устройства
override fun onSensorChanged(event: SensorEvent?) {
    // Проверка, что событие не null (данные датчика получены)
    event?.let { sensorEvent ->
        // Извлечение значений по осям X, Y, Z из объекта события
        val x = sensorEvent.values[0]
        val y = sensorEvent.values[1]
        val z = sensorEvent.values[2]

        // Обновление текста в TextView с использованием найденного элемента по ID.
        // Каждый раз при изменении датчика текст обновляется, отображая текущие
данные.
        findViewById<TextView>(R.id.accelerometerText).text = "X: $x\nY: $y\nZ:
$z"
    }
}

// Метод, вызываемый при изменении точности датчика. Используется редко, но
должен быть реализован
override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
    // Здесь можно добавить логику, реагирующую на изменение точности датчика,
например:
    // - Логирование ошибок при низкой точности.
    // - Изменение поведения приложения при переходе в высокоточный режим.
}
}

```

Рассмотрим основные компоненты, используемые в данном примере:

- **SensorManager** — служба Android, через которую регистрируются и управляются датчики. Она обеспечивает доступ к аппаратным датчикам и позволяет задавать параметры их работы.
- **Sensor.TYPE_ACCELEROMETER** — тип датчика, возвращающий ускорение по трём осям (X, Y, Z) с учётом гравитации.
- **SensorEventListener** — интерфейс, который необходимо реализовать для получения данных датчика.

Для управления жизненным циклом использовалось:

- `onResume()` — точка, где датчик активируется. Регистрация слушателя происходит здесь, чтобы данные считывались только тогда, когда пользователь взаимодействует с приложением.
- `onPause()` — точка, где датчик отключается. Отмена регистрации (`unregisterListener`) предотвращает расход батареи и утечки памяти.

Обработка данных датчика происходила с помощью:

- `onSensorChanged(event: SensorEvent?)` — основной метод, где происходят вычисления.
- `event?.let { }` — безопасная обработка события, исключая `NullPointerException`.
- `values[0], values[1], values[2]` — массив измерений датчика:
- `values[0]` — ускорение по оси X (влево/вправо).
- `values[1]` — ускорение по оси Y (вперёд/назад).
- `values[2]` — ускорение по оси Z (вверх/вниз).
- `findViewById<TextView>(R.id.accelerometerText)` — поиск текстового поля для отображения данных.

Также можно добавить проверку на доступность акселерометра:

```
if (accelerometer == null) {  
    Toast.makeText(this, "Акселерометр не доступен", Toast.LENGTH_SHORT).show()  
    finish() // Закрытие активности, если датчик отсутствует  
}
```

Пример 2: Использование датчика освещённости для автоматической смены темы.

// Активность, которая реализует `SensorEventListener` для получения данных с датчика освещённости

```
class LightSensorActivity : AppCompatActivity(), SensorEventListener {  
    // Менеджер датчиков, предоставляющий доступ к системным датчикам  
    private lateinit var sensorManager: SensorManager
```

// Переменная для хранения датчика освещённости. Может быть null, если датчик недоступен

```
    private var lightSensor: Sensor? = null
```

```

// Вызывается при создании активности
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_light_sensor) // Установка XML-макета

    // Получение системного сервиса датчиков
    sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager

    // Получение датчика освещённости. Если устройство не поддерживает его,
    lightSensor останется null
    lightSensor = sensorManager.getDefaultSensor(Sensor.TYPE_LIGHT)
}

// Вызывается при возобновлении активности. Здесь регистрируется датчик
override fun onResume() {
    super.onResume()
    // Проверка наличия датчика перед регистрацией слушателя
    lightSensor?.also { sensor ->
        // Регистрация слушателя с частотой SENSOR_DELAY_UI, оптимальной для
        обновления интерфейса
        sensorManager.registerListener(this, sensor,
        SensorManager.SENSOR_DELAY_UI)
    }
}

// Вызывается при паузе активности. Здесь отменяется регистрация датчика
override fun onPause() {
    super.onPause()
    // Отмена регистрации слушателя для экономии энергии и предотвращения
    утечек памяти
    sensorManager.unregisterListener(this)
}

// Вызывается при изменении данных датчика. Срабатывает многократно при
изменении освещённости
override fun onSensorChanged(event: SensorEvent?) {
    // Проверка, что событие не null (данные датчика получены)
    event?.let { sensorEvent ->
        // Извлечение уровня освещённости из массива значений датчика
        val lux = sensorEvent.values[0]

        // Поиск TextView для отображения данных.
        val textView = findViewById<TextView>(R.id.luxText)
        textView.text = "Освещённость: $lux люкс"
    }
}

```

```

// Логика смены темы в зависимости от уровня света
if (lux < 50) {
    // Темный режим: изменение цвета текста и фона экрана
    textView.setTextColor(Color.WHITE)
    window.decorView.setBackgroundColor(Color.BLACK)
} else {
    // Светлый режим: стандартные цвета
    textView.setTextColor(Color.BLACK)
    window.decorView.setBackgroundColor(Color.WHITE)
}
}
}

// Вызывается при изменении точности датчика. Пока не используется, но должен
быть реализован
override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
    // Здесь можно добавить логику, реагирующую на точность датчика, например:
    // - Вывод уведомления при низкой точности.
    // - Остановка обработки данных, если точность критически низкая.
}
}

```

- `Sensor.TYPE_LIGHT` — тип датчика, возвращающий уровень освещённости в люксах.
- `SENSOR_DELAY_UI` — задаёт частоту обновления, оптимальную для пользовательского интерфейса.
- `onSensorChanged()` обрабатывает данные датчика, изменяя цвет фона и текста в зависимости от уровня света.
- `window.decorView.setBackgroundColor()` — динамическое изменение цвета экрана, демонстрирующее реакцию на окружающую среду.

Датчики работают в фоновом потоке, поэтому обновления UI должны происходить через `runOnUiThread()` или корутины. Частота обновления (`SENSOR_DELAY_NORMAL`, `SENSOR_DELAY_UI`, `SENSOR_DELAY_GAME`, `SENSOR_DELAY_FASTEST`) влияет на точность и энергопотребление.

Приложения, использующие датчики, должны запрашивать разрешения в `AndroidManifest.xml` (например, `WAKE_LOCK` для длительной работы).

Жизненный цикл процессов, сохранение состояния приложения

Жизненный цикл активности (`onCreate`, `onResume`, `onPause`, `onDestroy`) играет центральную роль в работе с датчиками. Регистрация датчика через `registerListener()` должна происходить в `onResume()`, а отмена регистрации — в `onPause()`. Это связано с тем, что датчики продолжают считывать данные даже тогда, когда активность неактивна, что может привести к избыточному расходу батареи и утечкам памяти. Например, если датчик акселерометра зарегистрирован в `onCreate()`, но не отменён в `onPause()`, он будет продолжать генерировать события, даже если пользователь перешёл к другому экрану. Корректная интеграция с жизненным циклом гарантирует, что датчик будет активен только при необходимости.

Кроме того, важно учитывать, что некоторые датчики, такие как акселерометр или гироскоп, работают в фоновых потоках, и их данные требуют обработки в UI-потоке. Это требует синхронизации, которая реализуется через `Handler`, `runOnUiThread()` или корутины Kotlin. Например, в `onSensorChanged()` можно использовать `launch(Dispatchers.Main)` для обновления интерфейса, избегая блокировки основного потока.

Пример 1: Регистрация и отмена регистрации датчиков в жизненном цикле

// Активность, которая реализует `SensorEventListener` для получения данных с акселерометра

```
class MainActivity : AppCompatActivity(), SensorEventListener {
```

```
    // Менеджер датчиков, предоставляющий доступ к системным датчикам
```

```
    private lateinit var sensorManager: SensorManager
```

```
    // Переменная для хранения акселерометра. lateinit означает, что она будет  
    // инициализирована позже
```

```
    private lateinit var accelerometer: Sensor
```

```
    // Флаг, отслеживающий, зарегистрирован ли датчик в текущий момент
```

```
    private var isSensorRegistered = false
```

```
    // Метод, вызываемый при создании активности
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main) // Установка XML-макета
```

```

// Получение системного сервиса датчиков через Context.SENSOR_SERVICE
sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager

// Получение акселерометра.
accelerometer =
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
}

// Вызывается при возобновлении активности (например, после возвращения на
экран)
override fun onResume() {
    super.onResume()
    // Проверка, не зарегистрирован ли датчик ранее
    if (!isSensorRegistered) {
        // Регистрация слушателя с частотой SENSOR_DELAY_GAME,
        // оптимальной для игровых приложений.
        sensorManager.registerListener(this, accelerometer,
SensorManager.SENSOR_DELAY_GAME)
        isSensorRegistered = true // Обновление флага
    }
}

// Вызывается при паузе активности (например, при переходе к другому экрану)
override fun onPause() {
    super.onPause()
    // Проверка, зарегистрирован ли датчик, чтобы избежать лишних вызовов
    if (isSensorRegistered) {
        // Отмена регистрации слушателя для экономии энергии и предотвращения
утечек памяти
        sensorManager.unregisterListener(this)
        isSensorRegistered = false // Сброс флага
    }
}

// Вызывается при изменении данных датчика. Срабатывает многократно при
движении устройства
override fun onSensorChanged(event: SensorEvent?) {
    // Проверка, что событие не null (данные датчика получены)
    event?.let { sensorEvent ->
        // Поиск TextView в макете и обновление текста с текущими значениями
датчика
        findViewById<TextView>(R.id.sensorData).text =
            "Ускорение: ${sensorEvent.values[0]}, ${sensorEvent.values[1]},
${sensorEvent.values[2]}"
    }
}

```

```

    }

    // Вызывается при изменении точности датчика.
    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
        // Здесь можно добавить логику, реагирующую на изменение точности датчика,
        // например:
        // - Логирование ошибок при низкой точности.
        // - Изменение поведения приложения при переходе в высокоточный режим.
    }
}

```

В данном примере использовались следующие основные компоненты:

- `SensorManager` — служба Android, через которую регистрируются и управляются датчики.
- `Sensor.TYPE_ACCELEROMETER` — тип датчика, возвращающий ускорение по трём осям (X, Y, Z) с учётом гравитации.
- `SensorEventListener` — интерфейс, реализующий методы `onSensorChanged()` и `onAccuracyChanged()` для получения данных.

А также следующие методы:

- `onResume()` — точка, где датчик регистрируется, если ещё не был активирован.
- `onPause()` — точка, где датчик отключается для экономии энергии и предотвращения ошибок.
- `isSensorRegistered` — флаг, предотвращающий повторную регистрацию.
- `onSensorChanged(event: SensorEvent?)` — основной метод, где происходят вычисления.
- `onAccuracyChanged(sensor: Sensor?, accuracy: Int)` — метод, реагирующий на изменения точности датчика.

Таким образом в примере был продемонстрирован базовый подход к работе с акселерометром, где флаг `isSensorRegistered` предотвращает повторную регистрацию.

Приложения с датчиками часто сталкиваются с необходимостью сохранять данные при изменении конфигурации (например, при смене ориентации

экрана). Для этого используется `ViewModel` , который сохраняет данные между пересозданиями активности, и `LiveData` , который позволяет наблюдать за изменениями и обновлять интерфейс.

Например, если приложение отслеживает уровень освещённости, `ViewModel` может хранить последнее значение `lux`, а `LiveData` — уведомлять интерфейс о его изменении. Это исключает повторную инициализацию датчика при каждом пересоздании активности и обеспечивает непрерывность пользовательского опыта.

Кроме того, `LiveData` интегрируется с `ViewModel`, создавая паттерн `MVVM`, который упрощает тестирование и поддержку кода. Например, если данные датчика используются в нескольких частях приложения, `ViewModel` позволяет централизованно управлять ими, избегая дублирования логики.

Пример 2: Сохранение состояния с помощью `ViewModel`

```
// ViewModel для хранения и обновления данных датчика
class SensorViewModel : ViewModel() {
    // Приватная MutableLiveData для хранения строки с данными акселерометра
    private val _accelerometerData = MutableLiveData<String>()

    // Открытая LiveData для наблюдения из UI.
    // Наблюдатель получит обновления, но не сможет напрямую изменить данные
    val accelerometerData: LiveData<String> = _accelerometerData

    // Метод для обновления данных. Принимает массив Float значений от датчика
    fun updateSensorData(values: FloatArray) {
        // postValue() безопасно обновляет данные в основном потоке,
        // даже если метод вызван из фонового потока
        _accelerometerData.postValue("X: ${values[0]}, Y: ${values[1]}, Z: ${values[2]}")
    }
}

// Активность, реализующая SensorEventListener для работы с акселерометром
class SensorActivity : AppCompatActivity(), SensorEventListener {
    // Менеджер датчиков, предоставляющий доступ к системным датчикам
    private lateinit var sensorManager: SensorManager

    // Акселерометр, полученный из SensorManager
    private lateinit var accelerometer: Sensor
```

```

// Получение ViewModel через by viewModels(),
// которая автоматически связывает ViewModel с жизненным циклом активности
private val viewModel by viewModels<SensorViewModel>()

// Метод создания активности
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_sensor) // Установка макета

    // Получение SensorManager через системный сервис
    sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager

    // Получение акселерометра. Если датчик недоступен,
    // вызовет ошибку, так как используется lateinit
    accelerometer =
        sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

    // Подписка на LiveData для обновления UI при изменении данных
    viewModel.accelerometerData.observe(this) { data ->
        // Обновление текста в TextView при каждом новом значении LiveData
        findViewById<TextView>(R.id.sensorData).text = data
    }
}

// Метод возобновления активности. Регистрация датчика происходит здесь
override fun onResume() {
    super.onResume()
    // Регистрация слушателя датчика с частотой SENSOR_DELAY_NORMAL
    sensorManager.registerListener(this, accelerometer,
        SensorManager.SENSOR_DELAY_NORMAL)
}

// Метод паузы активности. Отмена регистрации датчика происходит здесь
override fun onPause() {
    super.onPause()
    // Отмена регистрации слушателя для экономии энергии и предотвращения
    утечек
    sensorManager.unregisterListener(this)
}

// Вызывается при изменении данных датчика. Обрабатывает новые значения
override fun onSensorChanged(event: SensorEvent?) {
    // Проверка, что событие не null (данные датчика получены)
    event?.let { sensorEvent ->
        // Передача значений акселерометра в ViewModel для обработки
    }
}

```

```

        viewModel.updateSensorData(sensorEvent.values)
    }
}

override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
    // Здесь можно добавить логику, реагирующую на точность датчика.
}
}

```

В данном примере:

- ViewModel — хранит данные, не зависящие от жизненного цикла активности.
- LiveData — позволяет наблюдать за изменениями данных и обновлять UI при каждом обновлении.
- `postValue()` — безопасный метод обновления данных из фонового потока.
- `observe()` — подписка на изменения, автоматически обновляющая интерфейс.

Использование ViewModel и LiveData становится стандартом для разделения бизнес-логики и отрисовки. Например, если приложение записывает данные акселерометра в базу данных Room, ViewModel может содержать метод `updateSensorData()`, который принимает данные и передаёт их в DAO-объект. LiveData же будет уведомлять интерфейс о каждом изменении, чтобы отображать актуальные значения. Это снижает риск ошибок, связанных с жизненным циклом активности, и упрощает интеграцию с другими компонентами, такими как фрагменты или сервисы.

Частота обновления датчика (`SENSOR_DELAY_NORMAL`, `SENSOR_DELAY_GAME` и т.д.) влияет не только на точность данных, но и на энергоэффективность. Например, в приложении для измерения освещённости нет необходимости использовать `SENSOR_DELAY_FASTEST`, так как резкие изменения освещения редки и не требуют немедленной реакции. В то же время, игра с управлением через акселерометр будет чувствовать себя нестабильно, если использовать

SENSOR_DELAY_NORMAL, так как пользователь ожидает мгновенной реакции.

Кроме того, важно учитывать, что частота обновления датчика зависит от аппаратной поддержки устройства. Некоторые датчики могут не поддерживать все режимы, что требует проверки через `sensor.maximumRange` и `sensor.resolution`.

Корутины Kotlin: асинхронная обработка данных датчиков

Корутины в Kotlin — это легковесные потоки, которые позволяют выполнять асинхронные и параллельные задачи без блокировки UI-потока. Они особенно полезны при обработке данных датчиков, так как могут обрабатывать вычисления в фоновых потоках, освобождая UI-поток для отрисовки.

Пример 1: Обработка данных акселерометра через корутину.

```
// Активность, реализующая SensorEventListener для получения данных датчика
class SensorCoroutineActivity : AppCompatActivity(), SensorEventListener {
    // Менеджер датчиков, предоставляющий доступ к системным датчикам
    private lateinit var sensorManager: SensorManager

    // Акселерометр, полученный из SensorManager
    private lateinit var accelerometer: Sensor

    // Корутинный scope, привязанный к жизненному циклу активности.
    // MainScope() использует Dispatchers.Main, что безопасно для обновления UI
    private val scope = MainScope()

    // Метод создания активности
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_sensor) // Установка макета

        // Получение SensorManager через системный сервис
        sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager

        // Получение акселерометра. Если датчик отсутствует, вызовет ошибку (lateinit)
        accelerometer =
            sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

        // Запуск корутины в основном потоке для периодического обновления UI
```

```

scope.launch(Dispatchers.Main) {
    while (true) {
        // Бесконечный цикл обновления данных, задержка 100 мс между
итерациями.
        // Это ограничивает частоту обновления UI
        delay(100)
    }
}

// Метод возобновления активности. Регистрация датчика происходит здесь
override fun onResume() {
    super.onResume()
    // Регистрация слушателя датчика с частотой SENSOR_DELAY_NORMAL
    sensorManager.registerListener(this, accelerometer,
SensorManager.SENSOR_DELAY_NORMAL)
}

// Метод паузы активности. Отмена регистрации датчика
override fun onPause() {
    super.onPause()
    // Отмена регистрации слушателя датчика для экономии энергии
    sensorManager.unregisterListener(this)

    // Очистка корутинного scope, чтобы избежать утечек памяти
    scope.cancel()
}

// Вызывается при изменении данных датчика. Обработывает новые значения
override fun onSensorChanged(event: SensorEvent?) {
    // Проверка, что событие не null (данные датчика получены)
    event?.let { sensorEvent ->
        // Запуск новой корутины в основном потоке для обновления UI
        scope.launch(Dispatchers.Main) {
            // Поиск TextView и обновление текста с текущими данными датчика.
            findViewById<TextView>(R.id.sensorData).text = "X: ${sensorEvent.values[0]}, Y:
${sensorEvent.values[1]}"
        }
    }
}

// Вызывается при изменении точности датчика
override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) { }
}

```


Этот пример демонстрирует базовый подход к обработке данных акселерометра через корутины. Были использованы следующие основные методы:

- `MainScope()` — создаёт корутину, привязанную к жизненному циклу активности.
- `launch(Dispatchers.Main)` — выполняет задачи в основном потоке, безопасно обновляя UI.
- `delay(100)` — задаёт интервал обновления, избегая перегрузки.
- `scope.cancel()` — завершает корутину в `onPause()`, предотвращая утечки памяти.

Пример 2: Обработка данных с датчика освещённости через корутину и Room-базу данных

// Активность, реализующая `SensorEventListener` для получения данных датчика освещённости

```
class LightSensorActivity : AppCompatActivity(), SensorEventListener {
```

```
    // Менеджер датчиков, предоставляющий доступ к системным датчикам
```

```
    private lateinit var sensorManager: SensorManager
```

```
    // Датчик освещённости, который будет зарегистрирован для получения данных
```

```
    private lateinit var lightSensor: Sensor
```

```
    // Экземпляр базы данных, созданный через Room
```

```
    private lateinit var database: AppDatabase
```

```
    // DAO (Data Access Object) для взаимодействия с таблицей базы данных
```

```
    private lateinit var sensorDao: SensorDao
```

```
    // Корутиновый scope, привязанный к жизненному циклу активности
```

```
    private val scope = MainScope()
```

```
    // Метод создания активности
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.id.lightSensorLayout) // Установка макета
```

```
        // Получение SensorManager через системный сервис
```

```
        sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
```

```

        // Получение датчика освещённости. Если датчик отсутствует, вызовет ошибку
(lateinit)
        lightSensor = sensorManager.getDefaultSensor(Sensor.TYPE_LIGHT)

        // Создание Room-базы данных.
        // databaseBuilder() использует контекст, класс базы данных и имя файла базы
        database = Room.databaseBuilder(
            applicationContext,
            AppDatabase::class.java,
            "sensor-database"
        ).build()

        // Получение DAO для работы с таблицей SensorData
        sensorDao = database.sensorDao()

        // Запуск корутины в фоновом потоке (Dispatchers.IO) для записи начального
значения
        scope.launch(Dispatchers.IO) {
            // Пример: запись начального значения освещённости (0f) в базу данных
            sensorDao.insert(SensorData(lux = 0f))
        }
    }

    // Метод возобновления активности. Регистрация датчика происходит здесь
    override fun onResume() {
        super.onResume()
        // Регистрация слушателя датчика с частотой SENSOR_DELAY_UI,
        // оптимальной для обновления пользовательского интерфейса
        sensorManager.registerListener(this, lightSensor,
            SensorManager.SENSOR_DELAY_UI)
    }

    // Метод паузы активности. Отмена регистрации датчика и очистка корутин
    override fun onPause() {
        super.onPause()
        // Отмена регистрации слушателя датчика для экономии энергии
        sensorManager.unregisterListener(this)

        // Очистка корутинного scope, чтобы избежать утечек памяти
        scope.cancel()
    }

    // Вызывается при изменении данных датчика. Обрабатывает новые значения
    override fun onSensorChanged(event: SensorEvent?) {
        // Проверка, что событие не null (данные датчика получены)

```

```

event?.let { sensorEvent ->
    // Извлечение уровня освещённости из массива значений датчика
    val lux = sensorEvent.values[0]

    // Запуск корутины в фоновом потоке для записи данных в базу
    scope.launch(Dispatchers.IO) {
        // Сохранение текущего уровня освещённости в базу данных
        sensorDao.insert(SensorData(lux = lux))
    }
}

// Вызывается при изменении точности датчика.
override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) { }
}

```

- Dispatchers.IO — корутина работает в фоновом потоке, чтобы не блокировать UI.
- Room — локальная база данных для хранения данных датчика.
- insert(SensorData(...)) — запись данных в базу через DAO-интерфейс.
- onSensorChanged() — вызывает корутину для фоновой записи, избегая задержек в UI-потоке.

Корутины упрощают асинхронную обработку, особенно при взаимодействии с базами данных или сетью. Для длительных операций (Dispatchers.IO) рекомендуется использовать отдельный поток, чтобы не блокировать UI. MainScope() автоматически отменяется при завершении активности, но его стоит явно отменять в onPause() или onDestroy().

Многопоточность и синхронизация данных

Работа с датчиками часто требует многопоточности, так как данные приходят из системного драйвера в отдельном потоке. Для безопасного обновления UI или обработки данных необходимо использовать синхронизацию, чтобы избежать гонки данных и непредсказуемого поведения.

Пример 1: Синхронизация данных через Handler.

```

// Активность, которая реализует SensorEventListener для получения данных датчика
class SensorHandlerActivity : AppCompatActivity(), SensorEventListener {
    // Менеджер датчиков, предоставляющий доступ к системным датчикам

```

```

private lateinit var sensorManager: SensorManager

// Акселерометр, полученный из SensorManager
private lateinit var accelerometer: Sensor

// Handler, привязанный к основному потоку, для обновления UI
private val handler = Handler(Looper.getMainLooper())

// Метод создания активности
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_handler) // Установка макета

    // Получение SensorManager через системный сервис
    sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager

    // Получение акселерометра. Если датчик отсутствует, вызовет ошибку (lateinit)
    accelerometer =
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
}

// Метод возобновления активности. Регистрация датчика происходит здесь
override fun onResume() {
    super.onResume()
    // Регистрация слушателя датчика с частотой SENSOR_DELAY_GAME,
    // оптимальной для игровых приложений
    sensorManager.registerListener(sensorEventListener, accelerometer,
SensorManager.SENSOR_DELAY_GAME)
}

// Метод паузы активности. Отмена регистрации датчика
override fun onPause() {
    super.onPause()
    // Отмена регистрации слушателя датчика для экономии энергии
    sensorManager.unregisterListener(sensorEventListener)
}

// Слушатель датчика, который обрабатывает изменение данных
private val sensorEventListener = SensorEventListener { event ->
    // Извлечение данных из события датчика
    val x = event.values[0]
    val y = event.values[1]
    val z = event.values[2]

    // handler.post() гарантирует, что обновление UI происходит в основном потоке,

```

```

// даже если данные датчика пришли из фонового потока
handler.post {
    // Поиск TextView и обновление текста с текущими данными датчика
    // Для оптимизации лучше сохранить ссылку на TextView при инициализации
    findViewById<TextView>(R.id.handlerText).text = "X: $x\nY: $y\nZ: $z"
}
}
}

```

Здесь `SensorManager` предоставляет доступ к системным датчикам и управляет регистрацией слушателей. `Sensor.TYPE_ACCELEROMETER` — это тип датчика, возвращающий ускорение по трём осям (X, Y, Z). `Handler(Looper.getMainLooper())` создаёт `Handler`, работающий в основном (UI) потоке, что безопасно для обновления интерфейса, а `SensorEventListener` — анонимный объект, обрабатывающий данные датчика. Этот подход формирует основу для создания приложений, которые не только собирают данные с датчика, но и эффективно их обрабатывают и отображают.

Пример 2: Использование `ExecutorService` для параллельной обработки.

// Активность, которая использует `ExecutorService` для фоновой обработки данных датчика

```

class SensorExecutorActivity : AppCompatActivity() {
    // Менеджер датчиков, предоставляющий доступ к системным датчикам
    private lateinit var sensorManager: SensorManager

    // Датчик освещённости, полученный из SensorManager
    private lateinit var lightSensor: Sensor

    // Однопоточный исполнитель для фоновой обработки данных
    private val executor = Executors.newSingleThreadExecutor()

    // Метод создания активности
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_executor) // Установка макета

        // Получение SensorManager через системный сервис
        sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager

        // Получение датчика освещённости. Если датчик отсутствует, вызовет ошибку (lateinit)
    }
}

```

```

        lightSensor = sensorManager.getDefaultSensor(Sensor.TYPE_LIGHT)
    }

    // Метод возобновления активности. Регистрация датчика происходит здесь
    override fun onResume() {
        super.onResume()
        // Регистрация слушателя датчика с частотой SENSOR_DELAY_UI,
        // подходящей для обновления пользовательского интерфейса
        sensorManager.registerListener(sensorEventListener, lightSensor,
        SensorManager.SENSOR_DELAY_UI)
    }

    // Метод паузы активности. Отмена регистрации датчика и завершение
    ExecutorService
    override fun onPause() {
        super.onPause()
        // Отмена регистрации слушателя датчика для экономии энергии
        sensorManager.unregisterListener(sensorEventListener)

        // Завершение фонового потока.
        executor.shutdown()
    }

    // Слушатель датчика, обрабатывающий изменения уровня освещённости
    private val sensorEventListener = SensorEventListener { event ->
        // Извлечение уровня освещённости из события
        val lux = event.values[0]

        // Запуск фоновой задачи через ExecutorService
        executor.execute {
            // Форматирование данных в фоновом потоке
            val formattedLux = formatLux(lux)

            // runOnUiThread() гарантирует, что обновление интерфейса произойдёт в
            // основном потоке, так как Android не позволяет обновлять UI вне UI-потока
            runOnUiThread {
                // Обновление текста в TextView.
                findViewById<TextView>(R.id.executorText).text = "Освещённость:
                $formattedLux"
            }
        }
    }

    // Метод форматирования уровня освещённости в строку
    private fun formatLux(lux: Float): String {

```

```

return when {
    lux < 10 -> "Очень темно" // Очень низкий уровень света
    lux < 100 -> "Темно" // Низкий уровень света
    lux < 1000 -> "Среднее освещение" // Средний уровень света
    else -> "Ярко" // Высокий уровень света
}
}
}

```

Этот пример демонстрирует использование `ExecutorService` для обработки данных датчика и обновления UI. `ExecutorService` — обеспечивает асинхронную обработку данных, исключая блокировку UI-потока. `runOnUiThread()` — гарантирует обновление интерфейса в основном потоке. `formatLux()` — преобразует числовое значение освещённости в пользовательский текст. `onResume()` и `onPause()` — управляют регистрацией датчика и жизненным циклом `Executor`.

Таким образом, `Handler` и `ExecutorService` — классические инструменты многопоточной обработки в Android, а `runOnUiThread()` — безопасный способ обновления UI из фонового потока. Для сложных сценариев многопоточности можно использовать `CountDownLatch` или `synchronized` для предотвращения гонки данных. Работа с датчиками в Android требует тщательной интеграции нескольких ключевых аспектов: управления жизненным циклом, многопоточной обработки данных, энергоэффективности и сохранения состояния приложения. Эти элементы тесно связаны и должны быть синхронизированы, чтобы обеспечить стабильное функционирование приложения и эффективное использование ресурсов устройства.

Эмуляторы Android Studio предоставляют ограниченные возможности для тестирования датчиков, особенно при работе с физическими параметрами, такими как акселерометр или магнитный датчик. Например, уровень освещённости на эмуляторе невозможно проверить адекватно, так как он не имеет доступа к реальному окружению. Тестирование на реальных устройствах позволяет: убедиться в корректности данных, проверить, как

приложение ведёт себя при изменении ориентации, переходе в фоновый режим или переключении между активностями, а также оценить расход батареи, который может быть критичным для долгосрочных задач.